

PART 1 — ARCHITECTURE WALKTHROUGH

(Say this while pointing at the flowchart)

OPENING

"What we've built is a fully agentic, multi-layer credit underwriting assistant. The idea is to replicate — and partially automate — the work a human credit analyst would do when a small business applies for a working capital loan.

Instead of one monolithic model doing everything, we've broken the problem into six distinct layers, each with its own agents, tools, and responsibilities. Every layer reads from and writes into a single shared memory object — we called it AppState — which flows through the entire pipeline. That object is the spine of the system."

SHARED WORKING MEMORY (point to the right-side box in the diagram)

"Before we go layer by layer, notice this box on the right — Shared Working Memory. This is a typed Python TypedDict object managed by LangGraph. Every agent in every layer reads from it and appends to it. It holds things like source_jsons, trust_weights, fraud_signals, effective_metrics, risk_score, policy_flags, recommended_limit, and the credit_memo.

We used LangGraph's Annotated reducer pattern — that means when multiple agents run in parallel and both try to write to the same field, LangGraph knows whether to merge the dictionaries or concatenate the lists, rather than one agent overwriting the other. This is the architectural decision that makes true parallelism safe without any locking code."

LAYER 1 — DATA ACQUISITION AGENTS

(Point to the top blue box in the diagram)

"Layer 1 is six parallel agents — one per data source. Banking, GST/Tax, Bureau, Financials, Ledger, and KYC.

The tech here is Groq's free-tier API running Llama 3.3 70B. We picked Groq specifically because it's free, fast, has no data retention policy concerns, and Llama 3.3 70B is strong enough at structured extraction that we didn't need a paid model.

Each agent works the same way: it receives a raw text document — think OCR'd bank statement, a GSTR-3B filing, a bureau report — and a strict Pydantic schema. It calls the LLM and says 'extract these fields, return only valid JSON matching this schema.' If the output fails Pydantic validation, we feed the error back to the model and it gets up to two more attempts to self-correct. That's the retry loop you can see in base_agent.py — no human in the loop, the agent figures it out.

On top of the schema validation, we have a second layer of business rules — a stand-in for Great Expectations — that checks domain logic: cash deposit ratio above 50% is suspicious, bureau score below 650 is flagged, GST vintage under 12 months is a warning. These run after extraction and append ValidationIssues to the result.

For orchestration, we used LangGraph's StateGraph with a fan-out pattern: all six agents start from the START node simultaneously, each writes their result into source_jsons under their own key, and LangGraph's merge reducer combines them when they finish. The six Groq calls go out in parallel — so the wall clock time is one call, not six sequential ones.

For the demo data, we built a synthetic document generator with three scenarios: clean, noisy — simulating garbled OCR output — and fraud_risk, which seeds deliberate inconsistencies like bank turnover much higher than GST-declared turnover, high cash deposits, DPD on bureau, anchor concentration above 70%. This lets us test the fraud detection logic without needing real customer data."

LAYER 2 — CONTEXT GRAPH

(Point to the purple box)

"Layer 2 takes Layer 1's extracted JSON and does two things in parallel.

First, a graph writer that projects all the entities into Neo4j — our graph database. Company node, Director nodes, GSTEntity, BankAccount, BureauProfile, FinancialsSnapshot, LedgerSnapshot. The relationships between them — HAS_DIRECTOR, FILED_GST, HOLDS_ACCOUNT — are first-class edges in the graph, not foreign keys in a table.

Why a graph database here? Because the diagram's RELATED_PARTY relationship is a one-hop Cypher query: find all Companies that share a Director with this one. In a relational database that's a multi-join. In Neo4j it's ten lines. More importantly, we also detect shared bank accounts across supposedly independent applicants — that's a much stronger fraud signal than shared directorship — and again, it's a single graph traversal.

Second, in parallel with the graph write, a pure Python trust-weight calculator compares the four independent turnover claims against each other: GST versus Bank, Bank versus Financials, GST versus Ledger. The variance between each pair maps to a trust weight — below 5% variance gives 0.95, above 35% gives 0.20. These pairwise weights go into the shared state as trust_weights.pairwise."

LAYER 3 — TRIANGULATION ENGINE

(Point to the yellow/orange box — marked HARDEST LAYER)

"This is what the diagram itself calls the hardest layer, and it genuinely is. Three jobs, two of them sequential, one parallel.

First, trust aggregation: we collapse the three pairwise comparisons into one weight per source. GST's weight is the average of its trust scores across all pairs it appears in. Same for bank, financials, ledger. This gives us the trust_weights.aggregated dictionary — source → confidence score.

Second, effective metrics: we compute a confidence-weighted average turnover across all four sources, weighted by those aggregated trust scores. This is the single number the rest of the pipeline uses — not any one source's figure, but a reconciled view. From that we derive working capital gap using two named, documented methods — the Nayak Committee turnover method (20% of effective turnover, which is the standard Indian bank approach for SME WC limits up to 5 crore) cross-checked against an operating cycle calculation using debtor days and creditor days from the ledger. We take the lower of the two. Then repayment capacity — EBITDA minus estimated existing debt service — and a current DSCR.

Third, running in parallel with the metric computation: the fraud detection node. This combines source-level red flags (high cash deposits, bureau write-offs, GST compliance status) with the two Neo4j graph traversals from Layer 2 — related party exposure and shared banking instrument. The shared bank account signal is severity HIGH because legitimate group companies sometimes share directors, but they essentially never share a bank account with an unrelated applicant. That's a structuring pattern. The output is a fraud_signals list and a contradictions list, plus a rolled-up fraud_risk of low, medium, or high."

LAYER 4 — POLICY ENGINE / BRE

(Point to the pink box)

"Layer 4 is the Business Rules Engine — pure deterministic logic, no LLM, no DB calls. Five hard eligibility rules pulled straight from the diagram: DSCR above 1.25, GST vintage above 12 months, Bureau score above 700, no DPD worse than 30 days, anchor concentration below 70%.

One design decision worth calling out: if the data a rule needs is missing because an upstream agent failed to extract it, the rule fails. We don't skip it. Absence of evidence is not the same as evidence of absence in credit underwriting.

The output is a policy_decision — clear, deviation_required, or policy_reject. Zero failures is clear. One or two failures triggers deviation_required, which means the case needs credit committee escalation before disbursement. Three or more is a policy reject. This band structure means a single failed rule doesn't kill an otherwise strong case, but multiple failures do."

LAYER 5 — ML RISK SCORING

(Point to the green ML box)

"Layer 5 is a real trained machine learning model, not a rule lookup. We used LightGBM — a gradient boosting framework — trained on 4,000 synthetic rows. The feature schema was designed to be compatible with the Amex Default Prediction dataset structure, with 18 meaningful features derived from AppState — bureau score, DPD counts, DSCR, debt/equity, fraud risk encoding, cash deposit ratio and so on — plus 170 placeholder columns filled with column means so the vector shape matches Amex's 188-feature structure. When you want to replace the synthetic training data with real Amex data, it's a single CSV swap, no code changes.

The model outputs three things: PD — probability of default — which gets converted to a 1-to-83 risk score matching the Amex leaderboard convention, where 83 is the safest; LGD, loss given default, estimated from the net_worth-to-exposure ratio; and expected loss rate, which is PD times LGD. The model is pre-trained once and cached as a joblib file — no retraining on every request."

LAYER 6 — SANCTION / LIMIT + CREDIT MEMO

(Point to the teal box)

"Layer 6 closes the loop with two parallel jobs.

The limit optimiser applies five constraints in sequence, each one acting as a ceiling on the previous: the RBI policy cap of 20% of effective turnover, the working capital need we computed in Layer 3, the repayment capacity from EBITDA, a risk appetite multiplier derived from the ML risk score — Band A cases get 100% of the computed limit, Band E gets 20% — and finally an exposure headroom check so total new plus existing exposure doesn't exceed 40% of effective turnover. The recommended limit is the minimum across all five. The output traces which constraint was binding, so a credit analyst can see exactly why the limit came out where it did.

In parallel, the Credit Memo Agent makes the one remaining LLM call in this layer. The pattern here is template plus evidence injection — sometimes called RAG-lite — where we inject all the verified numbers from AppState directly into the Groq prompt. The model is explicitly instructed to use only those injected numbers, never to invent figures. The memo comes out as six structured prose paragraphs — applicant profile, financial triangulation summary, risk flags, policy outcome, ML score interpretation, and the recommended limit with rationale — written the way a real credit analyst would write it for a committee review."

PART 2 — LIVE DEMO SCRIPT

(Say this while running each layer in the browser)

BEFORE YOU START

Open the app at <http://localhost:5173>. You'll see the empty console with the pipeline rail showing six idle grey dots at the top.

"What you're looking at is a React frontend connected to a FastAPI backend. The backend is running LangGraph as the orchestration layer — it manages the agent execution order and the shared state. The six dots across the top represent the six data acquisition agents in Layer 1. Right now they're idle."

GENERATING A CASE

Select "Fraud-risk signals" from the dropdown, then click Generate case.

"I'm generating a synthetic applicant case. This creates six raw documents — the kind of text you'd get from OCR'ing a bank statement, or pulling a GSTR-3B filing, or running a bureau check. The fraud-risk scenario deliberately seeds inconsistencies: the bank statement will show a higher turnover than the GST return, there'll be DPD on the bureau report, cash deposits will be high.

You can see the raw text for each source in the collapsible section below. This is what the agents will actually read — unstructured text, not clean JSON."

RUNNING LAYER 1

Click Run Layer 1.

"The six Groq calls are going out in parallel right now. Each agent is reading its source document and trying to extract structured data matching its Pydantic schema. If it gets it wrong, the validation error goes back to the model for a self-correction attempt.

[Wait for completion]

Look at the agent cards now. Each one shows the key extracted values — bureau score, DSCR, cash deposit ratio, debtor days and so on. The colour of each card's badge tells us the validation status — teal is valid, yellow is valid with warnings, red means extraction failed or a business rule fired.

Notice the confidence bar under each card — that drops slightly with each retry the model needed. And the issues section shows which business rules triggered. On a fraud-risk case you should see warnings about cash deposit ratio being high, and probably a GST vintage or bureau flag."

RUNNING LAYER 2

Click Run Layer 2.

"Layer 2 is writing all those extracted entities into Neo4j in the background. The graph view on the right is pulling the live neighbourhood of this Company node from Neo4j — you can see the Director nodes, the GSTEntity, BankAccount, and the other snapshot nodes connected by their relationship types.

The trust-weight cards on the left are the important output here. Look at the GST versus Bank comparison — on a fraud-risk case this variance will be high, meaning the trust weight is low. That's telling us the two independently sourced turnover figures don't agree, which is an early flag."

RUNNING LAYER 3

Click Run Layer 3.

"This is the reconciliation step. Layer 3 took those low trust weights from Layer 2 and computed a confidence-weighted effective turnover — not taking any one source at face value, but blending them weighted by how much they agreed with the others.

Look at the effective metrics cards. The effective turnover is our best estimate of actual business scale. The working capital gap is computed using the Nayak method — 20% of turnover — cross-checked against the operating cycle calculation from debtor and creditor days.

Now look at the fraud signals panel. On this fraud-risk case you should see the turnover_inflation signal at HIGH severity — that's because bank-inferred turnover substantially exceeds GST-declared turnover, which is a classic window-dressing pattern. The contradictions list below it shows the raw numeric disagreements across source pairs."

RUNNING LAYER 4

Click Run Layer 4.

"Five hard rules, evaluated right now with no LLM. Pure deterministic logic.

[Wait for result]

See the decision banner. On a fraud-risk case this will likely say DEVIATION REQUIRED or POLICY REJECT. Look at the per-rule breakdown — each row shows you the actual extracted value versus the required threshold, and a tick or cross. The DSCR rule and the bureau score rule are the ones most likely to fail on a fraud-risk applicant."

RUNNING LAYER 5

Click Run Layer 5.

"The LightGBM model is running on the 18 features we derived from this case's AppState. No new LLM call — this is a pre-trained sklearn pipeline loaded from disk.

The gauge shows the risk score out of 83 — higher is safer. Below it you can see the probability of default as a percentage, the loss given default estimate, and the expected loss rate which is PD times LGD. That's the actual credit loss you'd expect in basis points per rupee lent, which is what pricing is built on."

RUNNING LAYER 6

Click Run Layer 6.

"Two things happening in parallel now — the limit optimiser and the credit memo agent.

[Wait for result]

Look at the constraint waterfall on the left. Five bars, each one a different ceiling. The recommended limit is whichever bar is shortest — the binding constraint is highlighted. On a fraud-risk case with a low risk score, C4 — the risk appetite multiplier — will likely be the binding one, scaling the limit down to 20 or 40 percent of what it would otherwise be.

On the right, that's a live LLM-generated credit memo. The model was given all the verified numbers from this pipeline run and instructed to write in credit analyst prose. Notice it cites actual figures — it's not allowed to invent anything. You can copy this and drop it straight into a credit system as the analyst's write-up."

CLOSING

"To summarise what just happened: an applicant document set went in one end, and a recommended sanction limit with a committee-ready credit memo came out the other, in under two minutes, without a human analyst reading a single document.

The system isn't designed to replace the credit analyst — the deviation_required band and the memo are both designed to be reviewed by a human. What it does is compress the time from document submission to a structured, evidence-backed recommendation from days to minutes.

The architecture is also genuinely modular — you could plug in a real Account Aggregator feed at Layer 1, replace the synthetic training data with a real bureau default dataset at Layer 5, or add Qdrant for document similarity search between cases without changing any other layer. Each layer only reads and writes the shared AppState — everything else is encapsulated."

QUESTIONS YOU MIGHT GET

"Why not use one big LLM to do everything?" "The agent-per-source pattern gives you independent validation of each document type, retries scoped to one source without re-processing everything else, and source-level confidence scores you can trace. A single big call would give you one JSON blob with no way to know which fields it was confident about versus hallucinating."

"Why Groq/Llama and not GPT-4?" "Free tier with no data retention policy concerns, which matters for financial data. Llama 3.3 70B is strong enough for structured extraction with schema enforcement. The model is a config parameter — swapping it is one line in the .env file."

"Why Neo4j specifically?" "The related-party and shared bank account checks are one-hop graph traversals. In a relational database those are multi-join queries that get expensive as your case volume grows. In a graph DB they're constant-time. The node types also map naturally to the credit domain — Company, Director, BankAccount are first-class entities, not rows in a join table."

"How accurate is the ML model?" "Right now it's trained on synthetic data so the absolute PD numbers aren't calibrated to real default rates. What it does correctly is rank applicants relative to each other — the features are real credit signals. The model is a plug-in component; when you have real labelled data, you replace the training CSV and retrain. The inference code doesn't change."

"What happens with the shared memory if an agent fails?" "The LangGraph reducers handle partial writes gracefully — if the banking agent fails, source_jsons.banking is null, the other five agents still write their results, and the downstream layers see the missing key and handle it explicitly. Layer 4's rules that depend on missing data fail the rule rather than skip it, which is the conservative behaviour you want in a credit system."

End of script